

Plan de Desarrollo Técnico: Guardian Agent (Judge & Compliance)

Este documento detalla la estrategia de implementación para el **Guardian Agent**, el componente responsable de la seguridad, cumplimiento y validación final dentro del ecosistema del "Agente Autónomo de Ventas y Reservas".¹

1. Arquitectura Recomendada: "Human-in-the-Loop Simulation" (Judge Pattern)

A diferencia de los otros agentes que generan contenido o ejecutan acciones, el Guardian opera bajo un patrón de **"Juez Auditor"**. Su función no es conversar, sino **evaluar** una acción propuesta antes de que esta llegue al usuario.

Principio de Diseño: "Trust, but Verify". El sistema asume que los agentes (Negotiator, Scheduler, Concierge) pueden alucinar o cometer errores; el Guardian es el cortafuegos que impide que esos errores sean visibles para el cliente.

Flujo de Validación (The Guardian Pipeline)

El Guardian no es un simple prompt; es un pipeline secuencial de 3 capas:

1. **Capa de Seguridad (Safety Layer):** Detecta inyecciones de prompt, toxicidad, o fuga de información sensible (PII).
2. **Capa de Hechos (Fact-Checking Layer):** Verifica que la "Acción Propuesta" coincida matemáticamente y lógicamente con la "Evidencia" (Output de herramientas o RAG).
 - *Ejemplo:* Si el Scheduler dice "Cita confirmada a las 4pm", el Guardian verifica que el JSON de la API del calendario diga **status: confirmed** y **time: 16:00**.²
3. **Capa de Políticas (Policy Layer):** Asegura que la negociación no viole reglas comerciales (ej: descuentos no autorizados).³

2. Estructura de Datos (Inputs & Outputs)

Para codificar esto en **LangGraph/LangChain**, necesitamos definir estrictamente las interfaces de entrada y salida mediante **Pydantic Models**. Esto es crucial para que el Orchestrator sepa cómo reaccionar ante un rechazo.

Input Schema (Lo que recibe el Guardian)

El Guardian necesita el contexto completo para juzgar. No basta con el mensaje final.

JSON

```
{
  "request_id": "req_55901",
  "source_agent": "Scheduler", // ¿Quién propone la acción?
  "customer_intent": "Reserva de cita", // ¿Qué quería el usuario?
  "conversation_history_summary": "Usuario pide cita para mañana, rechazó horario de 10am, aceptó 4pm.",
  "proposed_action": {
    "type": "SEND_MESSAGE",
    "content": "¡Listo! Tu cita ha quedado confirmada para el martes a las 4:00 PM. El precio es $50."
  },
  "evidence_context": {
    "tool_outputs": [
      {
        "tool_name": "calendar_api",
        "output": {"booking_id": "bk_999", "start_time": "2025-12-17T16:00:00", "status": "CONFIRMED"}
      }
    ],
    "rag_chunks": [], // Si vino del Knowledge Concierge
    "active_policies": ["pol_cancelacion_01", "pol_precios_2025"]
  }
}
```

Output Schema (El veredicto)

Utilizaremos **Structured Output** (Function Calling) para garantizar que el LLM devuelva un JSON parseable, no texto libre.

JSON

```
{
  "status": "APPROVED", // Enum: APPROVED | REJECTED | ESCALATE
  "risk_score": 0.1, // 0.0 a 1.0
  "feedback": "La fecha confirmada en el mensaje coincide con la API. El tono es adecuado.",
  "modifications": null, // Si es REJECTED, aquí sugiere el cambio
  "action_required": "NONE" // Si es ESCALATE, indica "NOTIFY_HUMAN"
}
```

3. Lógica de Evaluación (El "Cerebro" del Juez)

Estrategia de Prompting: Chain of Thought (CoT)

El prompt del sistema debe forzar al modelo a "pensar" paso a paso antes de emitir el veredicto JSON.

System Prompt Blueprint:

"Eres el Agente Guardián (Judge & Compliance) de una clínica dental. Tu trabajo es proteger a la empresa y al cliente.

Tus Instrucciones:

1. Analiza la Evidencia: Compara el `proposed_message` con los `tool_outputs`.
¿Coinciden los datos (fechas, precios, IDs)? 4
2. Revisa Políticas: ¿El mensaje promete algo prohibido en `active_policies`?
3. Evalúa el Tono: Debe ser profesional, empático y claro.

Reglas de Veredicto:

* Si el mensaje dice 'Confirmado' pero la herramienta dice 'Error' o 'Pendiente' -> REJECTED (Alucinación detectada). 5

- Si el precio en el mensaje (\$50) es diferente al precio en la base de datos (\$80) -> REJECTED.
- * Si el usuario parece enojado o amenaza legalmente -> ESCALATE. 6

Formato de Salida:

Debes generar SOLO un objeto JSON que cumpla con el esquema GuardianVerdict."

4. Roadmap de Implementación

Fase 1: Definición de Modelos Pydantic (Setup)

- Crear `class GuardianInput(BaseModel)` y `class GuardianVerdict(BaseModel)`.
- Esto servirá como "contrato" entre los nodos del grafo en LangGraph.

Fase 2: Lógica de Validación (Core)

- Implementar la función `guardian_node(state)` en el grafo.
- Esta función debe tomar el estado actual (que contiene el `last_message` del agente anterior) y construir el payload de validación.

Fase 3: Integración de Feedback Loop

- **Si es REJECTED:** El grafo no debe ir a `END`. Debe tener una arista condicional que regrese al `Orchestrator` o al agente original (ej: `Scheduler`) con el `feedback` del Guardián inyectado en el contexto.⁷
 - *Ejemplo de Feedback al Scheduler:* "El Guardián rechazó tu mensaje. Razón: Dijiste 4pm pero la API retornó 5pm. Corrige el mensaje."

- **Si es ESCALATE:** El grafo dirige a un nodo especial **HumanHandoff** que alerta al equipo de soporte.

Fase 4: Pruebas Unitarias (Adversarial Testing)

Crear tests específicos para intentar "engañar" al Guardián:

1. **Test de Alucinación:** Forzar al Scheduler a inventar un ID de reserva y ver si el Guardián lo bloquea.
2. **Test de Precio:** Forzar al Negotiator a ofrecer un descuento del 90% y verificar que el Guardián aplique la política de "Límite de descuento".

5. Tecnologías Sugeridas

Componente	Tecnología Recomendada	Justificación
LLM (Judge)	GPT-4o	Se requiere el máximo nivel de razonamiento para detectar inconsistencias sutiles. Modelos pequeños (GPT-3.5) fallan en comparaciones numéricas precisas.
Framework	LangGraph	Esencial para manejar el flujo cíclico (Agente -> Guardian -> Rechazo -> Agente). ⁸
Validation	Pydantic	Para forzar que la salida del LLM sea estructurada y no requiera regex para parsear.
Observabilidad	LangSmith	Crítico para auditar por qué el Guardián aprobó o rechazó una acción durante el desarrollo.